# 数值代数大作业上机报告

崔正平 2400010714

**摘要**

本次实验报告分别通过多重网格方法、Uzawa Iteration Method 和 Inexact Uzawa Iteration Method 求解交错网格上 MAC 格式离散的 Stokes 方程, 并分析三种方法在不同参数下的循环次数、CPU 时间和计算误差.

## 1 问题描述

### 1.1 Stokes 方程与交错网格上的 MAC 格式

考虑 Stokes 方程

$$\begin{cases} -\Delta \vec{\boldsymbol{u}} + \nabla p = \vec{\boldsymbol{F}}, & (x,y) \in (0,1)\times(0,1), \\ \operatorname{div} \vec{\boldsymbol{u}} = 0, & (x,y) \in (0,1)\times(0,1), \end{cases} \tag{1}$$

边界条件为

$$\begin{aligned} \frac{\partial u}{\partial \vec{\boldsymbol{n}}} &= b, \quad y = 0, & \frac{\partial u}{\partial \vec{\boldsymbol{n}}} &= t, \quad y = 1, \\ \frac{\partial v}{\partial \vec{\boldsymbol{n}}} &= l, \quad x = 0, & \frac{\partial v}{\partial \vec{\boldsymbol{n}}} &= r, \quad x = 1, \\ u &= 0, \quad x = 0,1, & v &= 0, \quad y = 0,1, \end{aligned}$$

其中 $\vec{\boldsymbol{u}} = (u,v)$ 为速度, $p$ 为压力, $\vec{\boldsymbol{F}} = (f,g)$ 为外力, $\vec{\boldsymbol{n}}$ 为外法向方向.

利用交错网格上的 MAC 格式 (见图 1)

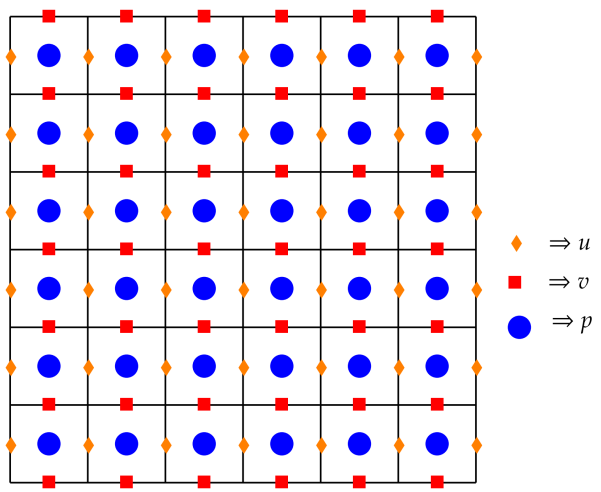


图 1: 交错网格上的 MAC 格式 (图片来源: 胡俊教授)

离散 Stokes 方程 (1) , 可得到如下线性方程组

$$\begin{pmatrix} \boldsymbol{A} & \boldsymbol{B} \\ \boldsymbol{B}^{\mathrm{T}} & \boldsymbol{O} \end{pmatrix} \begin{pmatrix} \boldsymbol{U} \\ \boldsymbol{P} \end{pmatrix} = \begin{pmatrix} \boldsymbol{F} \\ \boldsymbol{O} \end{pmatrix}. \tag{2}$$

## 1.2 数值算例

在区域 $\Omega = (0,1) \times (0,1)$ 上, 外力为

$$\begin{aligned} f(x,y) &= -4\pi^2(2\cos(2\pi x) - 1)\sin(2\pi y) + x^2, \\ g(x,y) &= 4\pi^2(2\cos(2\pi y) - 1)\sin(2\pi x). \end{aligned}$$

Stokes 方程 (1) 的真解为

$$\begin{aligned} u(x,y) &= (1 - \cos(2\pi x))\sin(2\pi y), \\ v(x,y) &= -(1 - \cos(2\pi y))\sin(2\pi x), \\ p(x,y) &= \frac{x^3}{3} - \frac{1}{12}. \end{aligned}$$

所有边界条件为

$$\begin{aligned} b(x) &= -2\pi(1 - \cos(2\pi x)), \\ t(x) &= 2\pi(1 - \cos(2\pi x)), \\ l(y) &= 2\pi(1 - \cos(2\pi y)), \\ r(y) &= -2\pi(1 - \cos(2\pi y)). \end{aligned}$$

## 1.3 大作业要求

**第一题:** 分别取 $N = 64, 128, 256, 512, 1024, 2048$ , 以 DGS 为磨光子, 用基于 V-cycle 的多重网格方法求解离散问题 (2) , 停机标准为 $\|\boldsymbol{r}_h\|_2/\|\boldsymbol{r}_0\|_2 \leqslant 10^{-8}$ , 对不同的 $\nu_1, \nu_2, L$ , 比较 V-cycle 的次数和 CPU 时间, 并计算误差

$$e_N = h\left(\sum_{j=1}^{N}\sum_{i=1}^{N-1}\left|u_{i,j-\frac{1}{2}} - u(x_i, y_{j-\frac{1}{2}})\right|^2 + \sum_{j=1}^{N-1}\sum_{i=1}^{N}\left|v_{i-\frac{1}{2},j} - v(x_{i-\frac{1}{2}}, y_j)\right|^2\right)^{\frac{1}{2}}.$$

**第二题:** 分别取 $N = 64, 128, 256, 512$ , 以 Uzawa Iteration Method 求解离散问题 (2) , 停机标准为 $\|\boldsymbol{r}_h\|_2/\|\boldsymbol{r}_0\|_2 \leqslant 10^{-8}$ , 并计算误差

$$e_N = h\left(\sum_{j=1}^{N}\sum_{i=1}^{N-1}\left|u_{i,j-\frac{1}{2}} - u(x_i, y_{j-\frac{1}{2}})\right|^2 + \sum_{j=1}^{N-1}\sum_{i=1}^{N}\left|v_{i-\frac{1}{2},j} - v(x_{i-\frac{1}{2}}, y_j)\right|^2\right)^{\frac{1}{2}}.$$

**第三题:** 分别取 $N = 64, 128, 256, 512, 1024, 2048$ , 以 Inexact Uzawa Iteration Method 为迭代法求解离散问题 (2) , 停机标准为 $\|\boldsymbol{r}_h\|_2/\|\boldsymbol{r}_0\|_2 \leqslant 10^{-8}$ , 其中以 V-cycle 多重网格方法为预条件子, 利用共轭梯度法求解每一步的子问题 $\boldsymbol{A}\boldsymbol{U}_{k+1} = \boldsymbol{F} - \boldsymbol{B}\boldsymbol{P}_k$ , 对不同的 $\alpha, \tau, \nu_1, \nu_2, L$ , 比较外循环的迭代次数和 CPU 时间, 并计算误差

$$e_N = h\left(\sum_{j=1}^{N}\sum_{i=1}^{N-1}\left|u_{i,j-\frac{1}{2}} - u(x_i, y_{j-\frac{1}{2}})\right|^2 + \sum_{j=1}^{N-1}\sum_{i=1}^{N}\left|v_{i-\frac{1}{2},j} - v(x_{i-\frac{1}{2}}, y_j)\right|^2\right)^{\frac{1}{2}}.$$

此外, 要求把所有误差结果以**表格**或者**图像**呈现, 要能体现**误差收敛阶**.

# 2 前置分析

## 2.1 交错网格上的 MAC 格式

关于 $u$ 的方程 $(h=1/N)$:

**(1)** 当 $2\leqslant i\leqslant N, 2\leqslant j\leqslant N-1$ 时,

$$-\frac{u_{i+1,j+\frac{1}{2}}-2u_{i,j+\frac{1}{2}}+u_{i-1,j+\frac{1}{2}}}{h^2}-\frac{u_{i,j+\frac{3}{2}}-2u_{i,j+\frac{1}{2}}+u_{i,j-\frac{1}{2}}}{h^2}+\frac{p_{i+\frac{1}{2},j+\frac{1}{2}}-p_{i-\frac{1}{2},j+\frac{1}{2}}}{h}=f_{i,j+\frac{1}{2}};$$

**(2)** 当 $2\leqslant i\leqslant N, j=1$ 时,

$$-\frac{u_{i,\frac{5}{2}}-u_{i,\frac{3}{2}}}{h^2}-\frac{b_{i,1}}{h}-\frac{u_{i+1,\frac{3}{2}}-2u_{i,\frac{3}{2}}+u_{i-1,\frac{3}{2}}}{h^2}+\frac{p_{i+\frac{1}{2},\frac{3}{2}}-p_{i-\frac{1}{2},\frac{3}{3}}}{h}=f_{i,\frac{3}{2}},$$

**(3)** 当 $2\leqslant i\leqslant N, j=N$ 时,

$$\frac{u_{i,N+\frac{1}{2}}-u_{i,N-\frac{1}{2}}}{h^2}-\frac{t_{i,N+1}}{h}-\frac{u_{i+1,N+\frac{1}{2}}-2u_{i,N+\frac{1}{2}}+u_{i-1,N+\frac{1}{2}}}{h^2}+\frac{p_{i+\frac{1}{2},N+\frac{1}{2}}-p_{i-\frac{1}{2},N+\frac{1}{2}}}{h}=f_{i,N+\frac{1}{2}};$$

**(4)** 当 $i=1,N+1,1\leqslant j\leqslant N$ 时, $u_{i,j+\frac{1}{2}}=0$ .

同理, 关于 $v$ 的方程 $(h=1/N)$:

**(1)** 当 $2\leqslant j\leqslant N, 2\leqslant i\leqslant N-1$ 时,

$$-\frac{v_{i+\frac{3}{2},j}-2v_{i+\frac{1}{2},j}+v_{i-\frac{1}{2},j}}{h^2}-\frac{v_{i+\frac{1}{2},j+1}-2v_{i+\frac{1}{2},j}+v_{i+\frac{1}{2},j-1}}{h^2}+\frac{p_{i+\frac{1}{2},j+\frac{1}{2}}-p_{i+\frac{1}{2},j-\frac{1}{2}}}{h}=g_{i+\frac{1}{2},j};$$

**(2)** 当 $2\leqslant j\leqslant N, i=1$ 时,

$$-\frac{v_{\frac{5}{2},j}-v_{\frac{3}{2},j}}{h^2}-\frac{l_{1,j}}{h}-\frac{v_{\frac{3}{2},j+1}-2v_{\frac{3}{2},j}+v_{\frac{3}{2},j-1}}{h^2}+\frac{p_{\frac{3}{2},j+\frac{1}{2}}-p_{\frac{3}{2},j-\frac{1}{2}}}{h}=g_{\frac{3}{2},j};$$

**(3)** 当 $2\leqslant j\leqslant N, i=N$ 时,

$$\frac{v_{N+\frac{1}{2},j}-v_{N-\frac{1}{2},j}}{h^2}-\frac{r_{N+1,j}}{h}-\frac{v_{N+\frac{1}{2},j+1}-2v_{N+\frac{1}{2},j}+v_{N+\frac{1}{2},j-1}}{h^2}+\frac{p_{N+\frac{1}{2},j+\frac{1}{2}}-p_{N+\frac{1}{2},j-\frac{1}{2}}}{h}=g_{N+\frac{1}{2},j};$$

**(4)** 当 $j=1,N+1,1\leqslant i\leqslant N$ 时, $v_{i+\frac{1}{2},j}=0$ .

另外有不可压条件 $(1\leqslant i\leqslant N, 1\leqslant j\leqslant N)$ :

$$\frac{u_{i+1,j+\frac{1}{2}}-u_{i,j+\frac{1}{2}}}{h}+\frac{v_{i+\frac{1}{2},j+1}-v_{i+\frac{1}{2},j}}{h}=0.$$

将上述等式整理可得如下线性方程组

$$\begin{pmatrix}\boldsymbol{A} & \boldsymbol{B}\\ \boldsymbol{B}^{\mathrm{T}} & \boldsymbol{O}\end{pmatrix}\begin{pmatrix}\boldsymbol{U}\\ \boldsymbol{P}\end{pmatrix}=\begin{pmatrix}\boldsymbol{F}\\ \boldsymbol{O}\end{pmatrix},$$

其中

$$\boldsymbol{U}=\begin{pmatrix}\boldsymbol{U}_u\\ \boldsymbol{U}_v\end{pmatrix}, \boldsymbol{F}=\begin{pmatrix}\boldsymbol{F}_f\\ \boldsymbol{F}_g\end{pmatrix},\quad \boldsymbol{U}_u,\boldsymbol{U}_v,\boldsymbol{F}_f,\boldsymbol{F}_g\in\mathbb{R}^{N(N-1)}, \boldsymbol{P}\in\mathbb{R}^{N^2},$$

$$\boldsymbol{A}=\begin{pmatrix}\boldsymbol{A}_u & \\ & \boldsymbol{A}_v\end{pmatrix}, \boldsymbol{B}=\begin{pmatrix}\boldsymbol{B}_u\\ \boldsymbol{B}_v\end{pmatrix},\quad \boldsymbol{A}_u,\boldsymbol{A}_v\in\mathbb{R}^{N(N-1)\times N(N-1)}, \boldsymbol{B}_u,\boldsymbol{B}_v\in\mathbb{R}^{N(N-1)\times N^2},$$

具体形式为:

$$\boldsymbol{U}_u=(u_{2,\frac{3}{2}},u_{3,\frac{3}{2}},\cdots,u_{N,\frac{3}{2}},u_{2,\frac{5}{2}},u_{3,\frac{5}{2}},\cdots,u_{N,\frac{5}{2}},\cdots,u_{2,N+\frac{1}{2}},u_{3,N+\frac{1}{2}},\cdots,u_{N,N+\frac{1}{2}})^{\mathrm{T}},$$

$$
\begin{aligned}
\boldsymbol{U}_v =& (v_{\frac{3}{2},2}, v_{\frac{5}{2},2}, \cdots, v_{N+\frac{1}{2},2}, v_{\frac{3}{2},3}, v_{\frac{5}{2},3}, \cdots, v_{N+\frac{1}{2},3}, \cdots, v_{\frac{3}{2},N}, v_{\frac{5}{2},N}, \cdots, v_{N+\frac{1}{2},N})^{\mathrm{T}}, \\
\boldsymbol{F}_f =& (f_{2,\frac{3}{2}} + \frac{b_{2,1}}{h}, f_{3,\frac{3}{2}} + \frac{b_{3,1}}{h}, \cdots, f_{N,\frac{3}{2}} + \frac{b_{N,1}}{h}, f_{2,\frac{5}{2}}, f_{3,\frac{5}{2}}, \cdots, f_{N,\frac{5}{2}}, \\
& \cdots, f_{2,N+\frac{1}{2}} + \frac{t_{2,N+1}}{h}, f_{3,N+\frac{1}{2}} + \frac{t_{3,N+1}}{h}, \cdots, f_{N,N+\frac{1}{2}} + \frac{t_{N,N+1}}{h})^{\mathrm{T}}, \\
\boldsymbol{F}_g =& (g_{\frac{3}{2},2} + \frac{l_{1,2}}{h}, g_{\frac{5}{2},2}, \cdots, g_{N+\frac{1}{2},2} + \frac{r_{N+1,2}}{h}, g_{\frac{3}{2},3} + \frac{l_{1,3}}{h}, g_{\frac{5}{2},3}, \cdots, g_{N+\frac{1}{2},3} + \frac{r_{N+1,3}}{h}, \\
& \cdots, g_{\frac{3}{2},N} + \frac{l_{1,N}}{h}, g_{\frac{5}{2},N}, \cdots, g_{N+\frac{1}{2},N} + \frac{r_{N+1,N}}{h})^{\mathrm{T}}, \\
\boldsymbol{P} =& (p_{\frac{3}{2},\frac{3}{2}}, p_{\frac{5}{2},\frac{3}{2}}, \cdots, p_{N+\frac{1}{2},\frac{3}{2}}, p_{\frac{3}{2},\frac{5}{2}}, p_{\frac{5}{2},\frac{5}{2}}, \cdots, p_{N+\frac{1}{2},\frac{5}{2}}, \cdots, p_{\frac{3}{2},N+\frac{1}{2}}, p_{\frac{5}{2},N+\frac{1}{2}}, \cdots, p_{N+\frac{1}{2},N+\frac{1}{2}})^{\mathrm{T}},
\end{aligned}
$$

$$
\boldsymbol{A}_u = \frac{1}{h^2}\begin{pmatrix}
\boldsymbol{X} & -\boldsymbol{I}_{N-1} & & & & \\
-\boldsymbol{I}_{N-1} & \boldsymbol{Y} & -\boldsymbol{I}_{N-1} & & & \\
& -\boldsymbol{I}_{N-1} & \boldsymbol{Y} & -\boldsymbol{I}_{N-1} & & \\
& & \ddots & \ddots & \ddots & \\
& & & -\boldsymbol{I}_{N-1} & \boldsymbol{Y} & -\boldsymbol{I}_{N-1} \\
& & & & -\boldsymbol{I}_{N-1} & \boldsymbol{X}
\end{pmatrix}, \boldsymbol{B}_u = \frac{1}{h}\mathrm{diag}(\boldsymbol{S}, \boldsymbol{S}, \cdots, \boldsymbol{S}),
$$

$$
\boldsymbol{A}_v = \frac{1}{h^2}\begin{pmatrix}
\boldsymbol{Z} & -\boldsymbol{I}_N & & & & \\
-\boldsymbol{I}_N & \boldsymbol{Z} & -\boldsymbol{I}_N & & & \\
& -\boldsymbol{I}_N & \boldsymbol{Z} & -\boldsymbol{I}_N & & \\
& & \ddots & \ddots & \ddots & \\
& & & -\boldsymbol{I}_N & \boldsymbol{Z} & -\boldsymbol{I}_N \\
& & & & -\boldsymbol{I}_N & \boldsymbol{Z}
\end{pmatrix}, \boldsymbol{B}_v = \frac{1}{h}\begin{pmatrix}
-\boldsymbol{I}_N & \boldsymbol{I}_N & & \\
& -\boldsymbol{I}_N & \boldsymbol{I}_N & \\
& & \ddots & \ddots \\
& & -\boldsymbol{I}_N & \boldsymbol{I}_N
\end{pmatrix},
$$

$$
\boldsymbol{X} = \begin{pmatrix}
3 & -1 & & & & \\
-1 & 3 & -1 & & & \\
& -1 & 3 & -1 & & \\
& & \ddots & \ddots & \ddots & \\
& & & -1 & 3 & -1 \\
& & & & -1 & 3
\end{pmatrix}, \quad \boldsymbol{Y} = \begin{pmatrix}
4 & -1 & & & & \\
-1 & 4 & -1 & & & \\
& -1 & 4 & -1 & & \\
& & \ddots & \ddots & \ddots & \\
& & & -1 & 4 & -1 \\
& & & & -1 & 4
\end{pmatrix},
$$

$$
\boldsymbol{Z} = \begin{pmatrix}
3 & -1 & & & & \\
-1 & 4 & -1 & & & \\
& -1 & 4 & -1 & & \\
& & \ddots & \ddots & \ddots & \\
& & & -1 & 4 & -1 \\
& & & & -1 & 3
\end{pmatrix}, \quad \boldsymbol{S} = \begin{pmatrix}
-1 & 1 & & & \\
& -1 & 1 & & \\
& & \ddots & \ddots & \\
& & & -1 & 1
\end{pmatrix}.
$$

## 2.2 $\boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{-1}\boldsymbol{B}$ 的特征值

我们断言: $\boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{-1}\boldsymbol{B}$ 的所有特征值为 0 (1 重) 和 1 ($N^2-1$ 重) .

事实上, 先取所有分量全相等的向量 $\boldsymbol{P} \in \mathbb{R}^{N^2}$ , 则 $\boldsymbol{B}\boldsymbol{P} = \boldsymbol{0}$ , 故 $\boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{-1}\boldsymbol{B}\boldsymbol{P} = \boldsymbol{0}$ , 从而特征值 $\lambda = 0$ 的重数至少为 1 .

再分析特征值 $\lambda = 1$ 的重数. 由于 $\boldsymbol{A}^{-1}\boldsymbol{B}\boldsymbol{B}^{\mathrm{T}}$ 和 $\boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{-1}\boldsymbol{B}$ 的非零特征值 (计重数) 完全相同, 故只需分析 $\boldsymbol{A}^{-1}\boldsymbol{B}\boldsymbol{B}^{\mathrm{T}}$ 的特征值 $\lambda = 1$ 的重数.

设向量 $\boldsymbol{P}\in\mathbb{R}^{N^2}$ 满足: $\boldsymbol{A}^{-1}\boldsymbol{B}\boldsymbol{B}^{\mathrm{T}}\boldsymbol{P}=\boldsymbol{P}$ , 这等价于 $\boldsymbol{B}\boldsymbol{B}^{\mathrm{T}}\boldsymbol{P}=\boldsymbol{A}\boldsymbol{P}$ , 也即 $(\boldsymbol{A}-\boldsymbol{B}\boldsymbol{B}^{\mathrm{T}})\boldsymbol{P}=\boldsymbol{0}$ . 通过计算可得

$$\boldsymbol{A}-\boldsymbol{B}\boldsymbol{B}^{\mathrm{T}}=$$

$$\begin{pmatrix}
\boldsymbol{I}_{N-1} & -\boldsymbol{I}_{N-1} & & & & & \boldsymbol{S} & & & & & \\
-\boldsymbol{I}_{N-1} & 2\boldsymbol{I}_{N-1} & -\boldsymbol{I}_{N-1} & & & & -\boldsymbol{S} & \boldsymbol{S} & & & & \\
 & -\boldsymbol{I}_{N-1} & 2\boldsymbol{I}_{N-1} & -\boldsymbol{I}_{N-1} & & & & -\boldsymbol{S} & \boldsymbol{S} & & & \\
 & & \ddots & \ddots & \ddots & & & & \ddots & \ddots & & \\
 & & & -\boldsymbol{I}_{N-1} & 2\boldsymbol{I}_{N-1} & -\boldsymbol{I}_{N-1} & & & & -\boldsymbol{S} & \boldsymbol{S} & \\
 & & & & -\boldsymbol{I}_{N-1} & \boldsymbol{I}_{N-1} & & & & & -\boldsymbol{S} & \\
\boldsymbol{S}^{\mathrm{T}} & -\boldsymbol{S}^{\mathrm{T}} & & & & & \boldsymbol{Z}-2\boldsymbol{I}_N & & & & & \\
 & \boldsymbol{S}^{\mathrm{T}} & -\boldsymbol{S}^{\mathrm{T}} & & & & & \boldsymbol{Z}-2\boldsymbol{I}_N & & & & \\
 & & \boldsymbol{S}^{\mathrm{T}} & -\boldsymbol{S}^{\mathrm{T}} & & & & & \boldsymbol{Z}-2\boldsymbol{I}_N & & & \\
 & & & \ddots & \ddots & & & & & \ddots & & \\
 & & & & \boldsymbol{S}^{\mathrm{T}} & -\boldsymbol{S}^{\mathrm{T}} & & & & & & \boldsymbol{Z}-2\boldsymbol{I}_N
\end{pmatrix},$$

因此不难验证 $(\boldsymbol{A}-\boldsymbol{B}\boldsymbol{B}^{\mathrm{T}})\boldsymbol{B}=0$ (这里用到了 $\boldsymbol{Z}=\boldsymbol{S}^{\mathrm{T}}\boldsymbol{S}+2\boldsymbol{I}_N$) , 而易知 $\boldsymbol{B}$ 的列秩为 $N^2-1$ , 从而可得 $\boldsymbol{A}-\boldsymbol{B}\boldsymbol{B}^{\mathrm{T}}$ 的核空间至少 $N^2-1$ 维, 故 $\boldsymbol{A}^{-1}\boldsymbol{B}\boldsymbol{B}^{\mathrm{T}}$ 的特征值 $\lambda=1$ 的重数至少为 $N^2-1$ , 于是 $\boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{-1}\boldsymbol{B}$ 的特征值 $\lambda=1$ 的重数至少为 $N^2-1$ .

又 $\boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{-1}\boldsymbol{B}$ 为 $N^2$ 阶方阵, 因此 $\boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{-1}\boldsymbol{B}$ 的所有特征值恰为 0 (1 重) 和 1 ($N^2-1$ 重) .

## 2.3 基于 V-cycle 的多重网格方法

### 2.3.1 算法原理

**(1)** 在 $\Omega^h$ 上, 以 $\boldsymbol{u}_h^{(0)}$ 为初值, 用 G-S 迭代法, 对 $\boldsymbol{A}_h\boldsymbol{U}_h=\boldsymbol{F}_h$ 迭代 $\nu_1$ 次, 得到 $\boldsymbol{u}_h^{(\nu_1)}$ , 并计算残量

$$\boldsymbol{r}_h=\boldsymbol{F}_h-\boldsymbol{A}_h\boldsymbol{u}_h^{(\nu_1)};$$

**(2)** 将残量 $\boldsymbol{r}_h$ 限制在网格 $\Omega^{2h}$ 上, 即

$$\boldsymbol{F}_{2h}=\boldsymbol{I}_h^{2h}\boldsymbol{r}_h;$$

**(3)** 在 $\Omega^{2h}$ 上, 以 $\boldsymbol{u}_{2h}^{(0)}=\boldsymbol{0}$ 为初值, 用 G-S 迭代法, 对 $\boldsymbol{A}_{2h}\boldsymbol{U}_{2h}=\boldsymbol{F}_{2h}$ 迭代 $\nu_1$ 次, 得到 $\boldsymbol{u}_{2h}^{(\nu_1)}$ , 并计算残量

$$\boldsymbol{r}_{2h}=\boldsymbol{F}_{2h}-\boldsymbol{A}_{2h}\boldsymbol{u}_{2h}^{(\nu_1)};$$

**(4)** 将残量 $\boldsymbol{r}_{2h}$ 限制在网格 $\Omega^{4h}$ 上, 即

$$\boldsymbol{F}_{4h}=\boldsymbol{I}_{2h}^{4h}\boldsymbol{r}_{2h};$$

**(5)** 在 $\Omega^{4h}$ 上, 以 $\boldsymbol{u}_{4h}^{(0)}=\boldsymbol{0}$ 为初值, 用 G-S 迭代法, 对 $\boldsymbol{A}_{4h}\boldsymbol{U}_{4h}=\boldsymbol{F}_{4h}$ 迭代 $\nu_1$ 次, 得到 $\boldsymbol{u}_{4h}^{(\nu_1)}$ , 并计算残量

$$\boldsymbol{r}_{4h}=\boldsymbol{F}_{4h}-\boldsymbol{A}_{4h}\boldsymbol{u}_{4h}^{(\nu_1)};$$

**(6)** 重复上述过程, 直到在 $\Omega^{Lh}$ 上, 以 $\boldsymbol{u}_{Lh}^{(0)}=\boldsymbol{0}$ 为初值, 用 G-S 迭代法, 对 $\boldsymbol{A}_{Lh}\boldsymbol{U}_{Lh}=\boldsymbol{F}_{Lh}$ 迭代 $\nu_1$ 次, 得到 $\boldsymbol{u}_{Lh}^{(\nu_1)}, L=2^r$ , 并计算残量

$$\boldsymbol{r}_{Lh}=\boldsymbol{F}_{Lh}-\boldsymbol{A}_{Lh}\boldsymbol{u}_{Lh}^{(\nu_1)};$$

**(7)** 将 $\boldsymbol{u}_{Lh}$ 提升到网格 $\Omega^{L/2,h}$ 上, 并更新 $\Omega^{L/2,h}$ 网格上的误差

$$\boldsymbol{u}_{L/2,h}^{(\nu_1+1)}=\boldsymbol{u}_{L/2,h}^{(\nu_1)}+\boldsymbol{I}_{Lh}^{L/2,h}\boldsymbol{u}_{Lh};$$

**(8)** 在 $\Omega^{L/2,h}$ 上, 以 $\boldsymbol{u}_{L/2,h}^{(\nu_1+1)}$ 为初值, 对

$$\boldsymbol{A}_{L/2,h}\boldsymbol{u}_{L/2,h}=\boldsymbol{F}_{L/2,h}$$

迭代 $\nu_2$ 次, 得到 $\boldsymbol{u}_{L/2,h}^{(\nu_1+\nu_2+1)}$ ;

**(9)** 将 $\boldsymbol{u}_{L/2,h}^{(\nu_1+\nu_2+1)}$ 提升到网格 $\Omega^{L/4,h}$ 上, 并更新 $\Omega^{L/4,h}$ 网格上的误差

$$\boldsymbol{u}_{L/4,h}^{(\nu_1+1)}=\boldsymbol{u}_{L/4,h}^{(\nu_1)}+\boldsymbol{I}_{L/2,h}^{L/4,h}\boldsymbol{u}_{L/2,h}^{(\nu_1+\nu_2+1)};$$

**(10)** 重复上述过程, 直到将 $\boldsymbol{u}_{2h}^{(\nu_1+\nu_2+1)}$ 提升到网格 $\Omega^h$ 上, 并更新 $\Omega^h$ 网格上的误差

$$\boldsymbol{u}_{h}^{(\nu_1+1)}=\boldsymbol{u}_{h}^{(\nu_1)}+\boldsymbol{I}_{2h}^{h}\boldsymbol{u}_{2h}^{(\nu_1+\nu_2+1)};$$

**(11)** 以 $\boldsymbol{u}_{h}^{(\nu_1+1)}$ 为初值, 在 $\Omega^h$ 上, 对

$$\boldsymbol{A}_h\boldsymbol{u}_h=\boldsymbol{F}_h$$

迭代 $\nu_2$ 次, 得到 $\boldsymbol{u}_{h}^{(\nu_1+\nu_2+1)}$ ;

**(12)** 计算残量

$$\boldsymbol{r}_h=\boldsymbol{F}_h-\boldsymbol{A}_h\boldsymbol{u}_h^{(\nu_1+\nu_2+1)},$$

若 $\|\boldsymbol{r}_h\|_2\leqslant$ 允许误差, 停止; 否则, 回到第二步.

### 2.3.2 具体算法实现

第一题中的基于 V-cycle 的多重网格方法如下:

```
function [U,V,count]=vcycle_1(F,G,N,eps,v_1,v_2,r)
    U_cell=cell(1,r+1);
    V_cell=cell(1,r+1);
    P_cell=cell(1,r+1);
    F_cell=cell(1,r+1);
    G_cell=cell(1,r+1);
    D_cell=cell(1,r+1);

    U_cell{1}=zeros(N-1,N);
    V_cell{1}=zeros(N,N-1);
    P_cell{1}=zeros(N,N);
    F_cell{1}=F;
    G_cell{1}=G;
    D_cell{1}=zeros(N,N);
```

```
    count=0;
    r_0=sqrt(norm(F,'fro')^2+norm(G,'fro')^2);

    for i=1:r+1
        n=N/2^(i-1);
        U_cell{i}=zeros(n-1,n);
        V_cell{i}=zeros(n,n-1);
        P_cell{i}=zeros(n,n);
    end

    [U_cell{1},V_cell{1},P_cell{1}]=DGS(U_cell{1},V_cell{1},P_cell{1},F_cell{1},G_cell
        {1},D_cell{1},N,v_1);

    while 1
        for i=2:r+1
            n=N/2^(i-1);
            U_cell{i}=zeros(n-1,n);
            V_cell{i}=zeros(n,n-1);
            P_cell{i}=zeros(n,n);
        end

        [F_1,G_1]=A_multiply(U_cell{1},V_cell{1},N);
        [F_2,G_2]=B_multiply(P_cell{1},N);
        D_1=BT_multiply(U_cell{1},V_cell{1},N);
        r_F=F_cell{1}-F_1-F_2;
        r_G=G_cell{1}-G_1-G_2;
        r_D=-D_1;
        r_1=sqrt(norm(r_F,'fro')^2+norm(r_G,'fro')^2+norm(r_D,'fro')^2);

        if r_1/r_0<=eps
            break
        end
        count=count+1;

        for i=1:r-1
            [F_cell{i+1},G_cell{i+1}]=restrict_UV(r_F,r_G,N/2^(i-1));
            D_cell{i+1}=restrict_P(r_D,N/2^(i-1));
            [U_cell{i+1},V_cell{i+1},P_cell{i+1}]=DGS(U_cell{i+1},V_cell{i+1},P_cell{i
                +1},F_cell{i+1},G_cell{i+1},D_cell{i+1},N/2^i,v_1);
            [F_1,G_1]=A_multiply(U_cell{i+1},V_cell{i+1},N/2^i);
            [F_2,G_2]=B_multiply(P_cell{i+1},N/2^i);
            D_1=BT_multiply(U_cell{i+1}, V_cell{i+1},N/2^i);
            r_F=F_cell{i+1}-F_1-F_2;
            r_G=G_cell{i+1}-G_1-G_2;
            r_D=D_cell{i+1}-D_1;
        end

        [F_cell{r+1},G_cell{r+1}]=restrict_UV(r_F,r_G,N/2^(r-1));
```

```
        D_cell{r+1}=restrict_P(r_D,N/2^(r-1));
        [U_cell{r+1},V_cell{r+1},P_cell{r+1}]=DGS(U_cell{r+1},V_cell{r+1},P_cell{r+1},
            F_cell{r+1},G_cell{r+1},D_cell{r+1},N/2^r,10);

        for i=r:-1:1
            [dU,dV]=lift_UV(U_cell{i+1},V_cell{i+1},N/2^i);
            dP=lift_P(P_cell{i+1},N/2^i);
            U_cell{i}=U_cell{i}+dU;
            V_cell{i}=V_cell{i}+dV;
            P_cell{i}=P_cell{i}+dP;
            [U_cell{i},V_cell{i},P_cell{i}]=DGS(U_cell{i},V_cell{i},P_cell{i},F_cell{i},
                G_cell{i},D_cell{i},N/2^(i-1),v_2);
        end
    end
    U=U_cell{1};
    V=V_cell{1};
end
```

第三题中的基于 V-cycle 的多重网格方法与此处类似, 故省略.

## 2.4 限制与提升算子

### 2.4.1 算子选取和分析

选取的限制与提升算子见图 2.

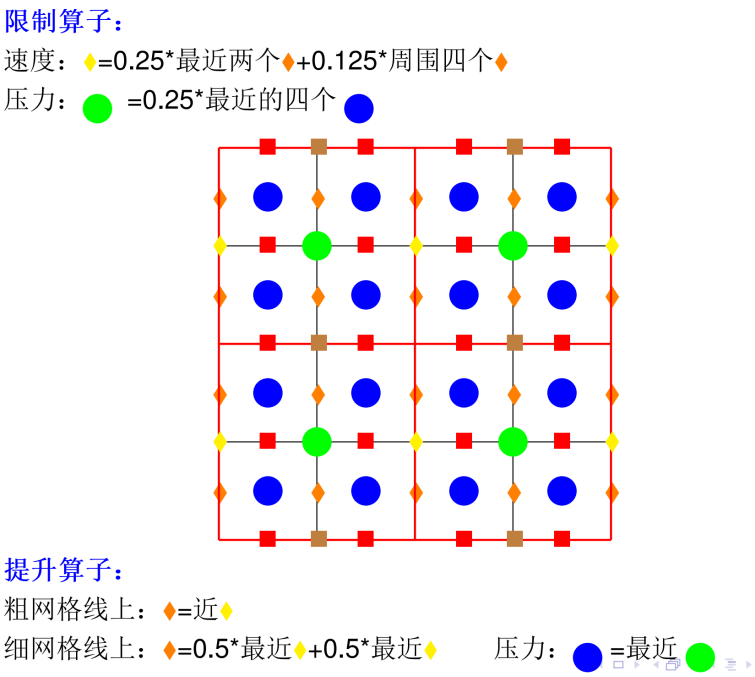


图 2: 限制与提升算子 (图片来源: 胡俊教授)

注意到在计算粗网格上的 $\boldsymbol{A}_{2h}, \boldsymbol{B}_{2h}$ 时, $\boldsymbol{A}_{2h}, \boldsymbol{B}_{2h}$ 应满足

$$\begin{pmatrix} \boldsymbol{A}_{2h} & \boldsymbol{B}_{2h} \\ \boldsymbol{B}_{2h}^{\mathrm{T}} & \boldsymbol{O} \end{pmatrix} = \boldsymbol{I}_h^{2h} \begin{pmatrix} \boldsymbol{A}_h & \boldsymbol{B}_h \\ \boldsymbol{B}_h^{\mathrm{T}} & \boldsymbol{O} \end{pmatrix} \boldsymbol{I}_{2h}^h,$$

但这样计算得到的 $\boldsymbol{A}_{2h}, \boldsymbol{B}_{2h}$ 形式会比较奇怪. 在实际应用中, 可以直接利用在粗网格上的 MAC 格式离散 Stokes 方程得到的系数矩阵去近似 $\boldsymbol{A}_{2h}, \boldsymbol{B}_{2h}$ , 以方便算法的实现.

### 2.4.2 具体算法实现

```
function [U_1,V_1]=restrict_UV(U,V,N)
    N_1=N/2;
    i_range=1:N_1-1;
    j_range=1:N_1;
    U_1=1/4*(U(2*i_range,2*j_range-1)+U(2*i_range,2*j_range))+1/8*(U(2*i_range-1,2*
        j_range-1)+U(2*i_range-1,2*j_range)+U(2*i_range+1,2*j_range-1)+U(2*i_range+1,2*
        j_range));
    V_1=1/4*(V(2*j_range-1,2*i_range)+V(2*j_range,2*i_range))+1/8*(V(2*j_range-1,2*
        i_range-1)+V(2*j_range,2*i_range-1)+V(2*j_range-1,2*i_range+1)+V(2*j_range,2*
        i_range+1));
end

function P_1=restrict_P(P,N)
    N_1=N/2;
    range=1:N_1;
    P_1=1/4*(P(2*range-1,2*range-1)+P(2*range-1,2*range)+P(2*range,2*range-1)+P(2*range
        ,2*range));
end

function [U_1,V_1]=lift_UV(U,V,N)
    N_1=2*N;
    U_1=zeros(N_1-1,N_1);
    V_1=zeros(N_1,N_1-1);

    range=1:N;
    U_1(1,2*range-1)=U(1,range)/2;
    U_1(1,2*range)=U(1,range)/2;
    U_1(N_1-1,2*range-1)=U(N-1,range)/2;
    U_1(N_1-1,2*range)=U(N-1,range)/2;
    V_1(2*range-1,1)=V(range,1)/2;
    V_1(2*range,1)=V(range,1)/2;
    V_1(2*range-1,N_1-1)=V(range,N-1)/2;
    V_1(2*range,N_1-1)=V(range,N-1)/2;

    i_range=1:N-1;
    j_range=1:N;
    U_1(2*i_range,2*j_range-1)=U(i_range,j_range);
    U_1(2*i_range,2*j_range)=U(i_range,j_range);
    V_1(2*j_range-1,2*i_range)=V(j_range,i_range);
```

```
    V_1(2*j_range,2*i_range)=V(j_range,i_range);

    i_range=1:N-2;
    U_1(2*i_range+1,2*j_range-1)=1/2*(U(i_range,j_range)+U(i_range+1,j_range));
    U_1(2*i_range+1,2*j_range)=1/2*(U(i_range,j_range)+U(i_range+1,j_range));
    V_1(2*j_range-1,2*i_range+1)=1/2*(V(j_range,i_range)+V(j_range,i_range+1));
    V_1(2*j_range,2*i_range+1)=1/2*(V(j_range,i_range)+V(j_range,i_range+1));
end

function P_1=lift_P(P,N)
    N_1=2*N;
    P_1=zeros(N_1,N_1);
    range=1:N;
    P_1(2*range-1,2*range-1)=P(range,range);
    P_1(2*range-1,2*range)=P(range,range);
    P_1(2*range,2*range-1)=P(range,range);
    P_1(2*range,2*range)=P(range,range);
end
```

这里采用了向量化并行加速, 以提高程序运行速度.

## 2.5 矩阵向量乘法

### 2.5.1 算法原理

为提高程序运行速度, 减小内存及存储开销, 在实现迭代法时不直接存储线性方程组 (2) 的系数矩阵, 并将 $\boldsymbol{U},\boldsymbol{P},\boldsymbol{F}$ 等向量以矩阵的形式存储, 即设

$$\widehat{\boldsymbol{U}}_u=\begin{pmatrix} u_{2,\frac{3}{2}} & u_{2,\frac{5}{2}} & \cdots & u_{2,N+\frac{1}{2}} \\ u_{3,\frac{3}{2}} & u_{3,\frac{5}{2}} & \cdots & u_{3,N+\frac{1}{2}} \\ \vdots & \vdots & \ddots & \vdots \\ u_{N,\frac{3}{2}} & u_{N,\frac{5}{2}} & \cdots & u_{N,N+\frac{1}{2}} \end{pmatrix},\quad \widehat{\boldsymbol{U}}_v=\begin{pmatrix} v_{\frac{3}{2},2} & v_{\frac{3}{2},3} & \cdots & v_{\frac{3}{2},N} \\ v_{\frac{5}{2},2} & v_{\frac{5}{2},3} & \cdots & v_{\frac{5}{2},N} \\ \vdots & \vdots & \ddots & \vdots \\ v_{N+\frac{1}{2},2} & v_{N+\frac{1}{2},3} & \cdots & v_{N+\frac{1}{2},N}, \end{pmatrix},$$

$$\widehat{\boldsymbol{P}}=\begin{pmatrix} p_{\frac{3}{2},\frac{3}{2}} & p_{\frac{3}{2},\frac{5}{2}} & \cdots & p_{\frac{3}{2},N+\frac{1}{2}} \\ p_{\frac{5}{2},\frac{3}{2}} & p_{\frac{5}{2},\frac{5}{2}} & \cdots & p_{\frac{5}{2},N+\frac{1}{2}} \\ \vdots & \vdots & \ddots & \vdots \\ p_{N+\frac{1}{2},\frac{3}{2}} & p_{N+\frac{1}{2},\frac{5}{2}} & \cdots & p_{N+\frac{1}{2},N+\frac{1}{2}} \end{pmatrix},\quad \widehat{\boldsymbol{F}}_f=\begin{pmatrix} f_{2,\frac{3}{2}}+\frac{b_{2,1}}{h} & f_{2,\frac{5}{2}} & \cdots & f_{2,N+\frac{1}{2}}+\frac{t_{2,N+1}}{h} \\ f_{3,\frac{3}{2}}+\frac{b_{3,1}}{h} & f_{3,\frac{5}{2}} & \cdots & f_{3,N+\frac{1}{2}}+\frac{t_{3,N+1}}{h} \\ \vdots & \vdots & \ddots & \vdots \\ f_{N,\frac{3}{2}}+\frac{b_{N,1}}{h} & f_{N,\frac{5}{2}} & \cdots & f_{N,N+\frac{1}{2}}+\frac{t_{N,N+1}}{h} \end{pmatrix},$$

$$\widehat{\boldsymbol{F}}_g=\begin{pmatrix} g_{\frac{3}{2},2}+\frac{l_{1,2}}{h} & g_{\frac{3}{2},3}+\frac{l_{1,3}}{h} & \cdots & g_{\frac{3}{2},N}+\frac{l_{1,N}}{h} \\ g_{\frac{5}{2},2} & g_{\frac{5}{2},3} & \cdots & g_{\frac{5}{2},N} \\ \vdots & \vdots & \ddots & \vdots \\ g_{N+\frac{1}{2},2}+\frac{r_{N+1,2}}{h} & g_{N+\frac{1}{2},3}+\frac{r_{N+1,3}}{h} & \cdots & g_{N+\frac{1}{2},N}+\frac{r_{N+1,N}}{h} \end{pmatrix}.$$

此时矩阵向量乘法 $\boldsymbol{AU},\boldsymbol{BP},\boldsymbol{B}^{\mathrm{T}}\boldsymbol{U}$ 中的 $\boldsymbol{A},\boldsymbol{B},\boldsymbol{B}^{\mathrm{T}}$ 可分别看作负的离散的 Laplace 算子 $-\Delta$ 、离散的梯度算子 $\nabla$ 和负的离散的散度算子 $-\nabla\cdot$ , 于是矩阵向量乘法就可直接对 $\widehat{\boldsymbol{U}}_u,\widehat{\boldsymbol{U}}_v,\widehat{\boldsymbol{P}}$ 中的元素进行运算, 而完全不依赖于 $\boldsymbol{A},\boldsymbol{B}$ 和 $\boldsymbol{B}^{\mathrm{T}}$ 的具体形式, 其中位于内部的形式一致的大量线性运算可通过向量化并行加速, 显著提高程序运行速度, 边界情形需特殊处理.

### 2.5.2 具体算法实现

```
function [F_1,G_1]=A_multiply(U,V,N)
    n=N^2;
    F_1=zeros(N-1,N); G_1=zeros(N,N-1);

    i_range=2:(N-2); j_range=2:(N-1);
    F_1(i_range,j_range)=(4*U(i_range,j_range)-U(i_range,j_range-1)-U(i_range,j_range+1)-U(i_range-1,j_range)-U(i_range+1,j_range))*n;
    G_1(j_range,i_range)=(4*V(j_range,i_range)-V(j_range-1,i_range)-V(j_range+1,i_range)-V(j_range,i_range-1)-V(j_range,i_range+1))*n;

    F_1(1,j_range)=(4*U(1,j_range)-U(1,j_range-1)-U(1,j_range+1)-U(2,j_range))*n;
    F_1(N-1,j_range)=(4*U(N-1,j_range)-U(N-1,j_range-1)-U(N-1,j_range+1)-U(N-2,j_range))*n;
    G_1(j_range,1)=(4*V(j_range,1)-V(j_range-1,1)-V(j_range+1,1)-V(j_range,2))*n;
    G_1(j_range,N-1)=(4*V(j_range,N-1)-V(j_range-1,N-1)-V(j_range+1,N-1)-V(j_range,N-2))*n;

    F_1(i_range,1)=(3*U(i_range,1)-U(i_range-1,1)-U(i_range+1,1)-U(i_range,2))*n;
    F_1(i_range,N)=(3*U(i_range,N)-U(i_range-1,N)-U(i_range+1,N)-U(i_range,N-1))*n;
    G_1(1,i_range)=(3*V(1,i_range)-V(1,i_range-1)-V(1,i_range+1)-V(2,i_range))*n;
    G_1(N,i_range)=(3*V(N,i_range)-V(N,i_range-1)-V(N,i_range+1)-V(N-1,i_range))*n;

    F_1(1,1)=(3*U(1,1)-U(2,1)-U(1,2))*n;
    F_1(N-1,1)=(3*U(N-1,1)-U(N-2,1)-U(N-1,2))*n;
    F_1(1,N)=(3*U(1,N)-U(1,N-1)-U(2,N))*n;
    F_1(N-1,N)=(3*U(N-1,N)-U(N-2,N)-U(N-1,N-1))*n;
    G_1(1,1)=(3*V(1,1)-V(1,2)-V(2,1))*n;
    G_1(1,N-1)=(3*V(1,N-1)-V(1,N-2)-V(2,N-1))*n;
    G_1(N,1)=(3*V(N,1)-V(N-1,1)-V(N,2))*n;
    G_1(N,N-1)=(3*V(N,N-1)-V(N,N-2)-V(N-1,N-1))*n;
end

function [F_1,G_1]=B_multiply(P,N)
    i_range=1:N-1; j_range=1:N;
    F_1(i_range,j_range)=(P(i_range+1,j_range)-P(i_range,j_range))*N;
    G_1(j_range,i_range)=(P(j_range,i_range+1)-P(j_range,i_range))*N;
end

function D_1=BT_multiply(U,V,N)
    D_1=zeros(N,N);

    range=2:N-1;
    D_1(range,range)=-(U(range,range)-U(range-1,range)+V(range,range)-V(range,range-1))*N;

    D_1(range,1)=-(U(range,1)-U(range-1,1)+V(range,1))*N;
    D_1(range,N)=-(U(range,N)-U(range-1,N)-V(range,N-1))*N;
```

```
    D_1(1,range)=-(U(1,range)+V(1,range)-V(1,range-1))*N;
    D_1(N,range)=-(-U(N-1,range)+V(N,range)-V(N,range-1))*N;

    D_1(1,1)=-(U(1,1)+V(1,1))*N;
    D_1(1,N)=-(U(1,N)-V(1,N-1))*N;
    D_1(N,1)=-(-U(N-1,1)+V(N,1))*N;
    D_1(N,N)=-(-U(N-1,N)-V(N,N-1))*N;
end
```

### 2.6 硬件配置及软件环境

- CPU: Intel(R) Core(TM) Ultra 5 125H.
- 内存: 32G.
- 软件: MATLAB R2023a.

## 3 第一题

### 3.1 数值方法

采用以 DGS 为磨光子, 基于 V-cycle 的多重网格方法, 其中 Distributive Gauss-Seidel (DGS) 迭代法定义如下 (A. Brandt & N. Dinar 1979): 给定初始值 $\boldsymbol{U}_0=(\boldsymbol{u}_0^{\mathrm{T}},\boldsymbol{v}_0^{\mathrm{T}})^{\mathrm{T}}$ 和 $\boldsymbol{P}_0$ , 令 $k=0,\boldsymbol{A}=\boldsymbol{D_A}-\boldsymbol{L_A}-\boldsymbol{U_A}$.
**1.** 用 Gauss-Seidel 迭代方法更新速度分量

$$\boldsymbol{U}_{k+\frac{1}{2}}=\boldsymbol{U}_k+(\boldsymbol{D_A}-\boldsymbol{L_A})^{-1}(\boldsymbol{F}-\boldsymbol{B}\boldsymbol{P}_k-\boldsymbol{A}\boldsymbol{U}_k);$$

**2.** 对内部单元 $(i,j)$ (四个顶点 $((i-1)h,(j-1)h),(ih,(j-1)h),(ih,jh),((i-1)h,jh),2\leqslant i,j\leqslant N-1)$ , 计算散度方程的残量

$$r_{i,j}=-\frac{u_{i,j-\frac{1}{2}}^{k+\frac{1}{2}}-u_{i-1,j-\frac{1}{2}}^{k+\frac{1}{2}}}{h}-\frac{v_{i-\frac{1}{2},j}^{k+\frac{1}{2}}-v_{i-\frac{1}{2},j-1}^{k+\frac{1}{2}}}{h},$$

并令 $\delta=\frac{r_{i,j}h}{4}$ ;
**3.** 更新内部单元速度:

$$\begin{aligned}u_{i-1,j-\frac{1}{2}}^{k+1}&=u_{i-1,j-\frac{1}{2}}^{k+\frac{1}{2}}-\delta,\quad u_{i,j-\frac{1}{2}}^{k+1}=u_{i,j-\frac{1}{2}}^{k+\frac{1}{2}}+\delta,\\ v_{i-\frac{1}{2},j-1}^{k+1}&=v_{i-\frac{1}{2},j-1}^{k+\frac{1}{2}}-\delta,\quad v_{i-\frac{1}{2},j}^{k+1}=v_{i-\frac{1}{2},j}^{k+\frac{1}{2}}+\delta;\end{aligned}$$

**4.** 更新内部单元压力:

$$\begin{aligned}p_{i-\frac{1}{2},j-\frac{1}{2}}^{k+1}&=p_{i-\frac{1}{2},j-\frac{1}{2}}^{k}+r_{i,j},\\ p_{i+\frac{1}{2},j-\frac{1}{2}}^{k+1}&=p_{i+\frac{1}{2},j-\frac{1}{2}}^{k}-\frac{r_{i,j}}{4},\quad p_{i-\frac{3}{2},j-\frac{1}{2}}^{k+1}=p_{i-\frac{3}{2},j-\frac{1}{2}}^{k}-\frac{r_{i,j}}{4},\\ p_{i-\frac{1}{2},j+\frac{1}{2}}^{k+1}&=p_{i-\frac{1}{2},j+\frac{1}{2}}^{k}-\frac{r_{i,j}}{4},\quad p_{i-\frac{1}{2},j-\frac{3}{2}}^{k+1}=p_{i-\frac{1}{2},j-\frac{3}{2}}^{k}-\frac{r_{i,j}}{4};\end{aligned}$$

**5.** 对边界单元 $(i,N)$ (四个顶点 $((i-1)h,(N-1)h),(ih,(N-1)h),(ih,Nh),((i-1)h,Nh),2\leqslant i\leqslant N-1)$ , 先计算散度方程的残量 $r_{i,N}$ , 计算公式同前, 并令 $\delta=\frac{r_{i,N}h}{3}$ , 速度更新如下:

$$u_{i-1,N-\frac{1}{2}}^{k+1}=u_{i-1,N-\frac{1}{2}}^{k+\frac{1}{2}}-\delta,\quad u_{i,N-\frac{1}{2}}^{k+1}=u_{i,N-\frac{1}{2}}^{k+\frac{1}{2}}+\delta,\quad v_{i-\frac{1}{2},N-1}^{k+1}=v_{i-\frac{1}{2},N-1}^{k+\frac{1}{2}}-\delta;$$

**6.** 更新边界单元 $(i,N)$ 的压力:

$$
\begin{aligned}
&p_{i-\frac{1}{2},N-\frac{1}{2}}^{k+1}=p_{i-\frac{1}{2},N-\frac{1}{2}}^{k}+r_{i,N},\\
&p_{i+\frac{1}{2},N-\frac{1}{2}}^{k+1}=p_{i+\frac{1}{2},N-\frac{1}{2}}^{k}-\frac{r_{i,N}}{3},\quad p_{i-\frac{3}{2},N-\frac{1}{2}}^{k+1}=p_{i-\frac{3}{2},N-\frac{1}{2}}^{k}-\frac{r_{i,N}}{3},\quad p_{i-\frac{1}{2},N-\frac{3}{2}}^{k+1}=p_{i-\frac{1}{2},N-\frac{3}{2}}^{k}-\frac{r_{i,N}}{3};
\end{aligned}
$$

**7.** 类似更新边界单元 $(N,j)$ 和 $(1,j),2\leqslant j\leqslant N-1$ 与 $(i,1),2\leqslant i\leqslant N-1$ 的速度和压力;

**8.** 对顶点单元 $(1,1)$ , 计算散度方程的残量 $r_{1,1}$ , 计算公式同前, 令 $\delta=\frac{r_{1,1}h}{2}$ , 更新速度如下:

$$
u_{1,\frac{1}{2}}^{k+1}=u_{1,\frac{1}{2}}^{k+\frac{1}{2}}+\delta,\quad v_{\frac{1}{2},1}^{k+1}=v_{\frac{1}{2},1}^{k+\frac{1}{2}}+\delta,
$$

更新压力如下:

$$
p_{\frac{1}{2},\frac{1}{2}}^{k+1}=p_{\frac{1}{2},\frac{1}{2}}^{k}+r_{1,1},\quad p_{\frac{3}{2},\frac{1}{2}}^{k+1}=p_{\frac{3}{2},\frac{1}{2}}^{k}-\frac{r_{1,1}}{2},\quad p_{\frac{1}{2},\frac{3}{2}}^{k+1}=p_{\frac{1}{2},\frac{3}{2}}^{k}-\frac{r_{1,1}}{2};
$$

**9.** 对其他顶点单元 $(1,N),(N,1)$ 和 $(N,N)$ , 类似更新速度和压力.

注意到对残量方程做提升或限制后, 得到的新线性方程组一般不会满足数值散度为 0 的条件, 此时待求解的线性方程组将形如

$$
\begin{pmatrix}\boldsymbol{A} & \boldsymbol{B}\\ \boldsymbol{B}^{\mathrm{T}} & \boldsymbol{O}\end{pmatrix}\begin{pmatrix}\boldsymbol{U}\\ \boldsymbol{P}\end{pmatrix}=\begin{pmatrix}\boldsymbol{F}\\ \boldsymbol{D}\end{pmatrix},\tag{3}
$$

其中 $\boldsymbol{D}=[d_{i,j}]$ 为负的数值散度, 因而在利用 DGS 迭代法求解线性方程组 (3) 时, 散度方程的残量 $r_{i,j}$ 将变为

$$
r_{i,j}=-\frac{u_{i,j-\frac{1}{2}}^{k+\frac{1}{2}}-u_{i-1,j-\frac{1}{2}}^{k+\frac{1}{2}}}{h}-\frac{v_{i-\frac{1}{2},j}^{k+\frac{1}{2}}-v_{i-\frac{1}{2},j-1}^{k+\frac{1}{2}}}{h}-d_{i,j},
$$

即 $\boldsymbol{r}=\boldsymbol{B}^{\mathrm{T}}\boldsymbol{U}_{k+\frac{1}{2}}-\boldsymbol{D}=-\nabla_h\cdot\boldsymbol{U}_{k+\frac{1}{2}}-\boldsymbol{D}$ .

## 3.2 具体算法实现

```
function [U,V,P]=DGS(U,V,P,F,G,D,N,m)
    h=1/N;
    for t=1:m
        [F_1,G_1]=A_multiply(U,V,N);
        [F_2,G_2]=B_multiply(P,N);
        F_3=F-F_1-F_2;
        G_3=G-G_1-G_2;
        [dU,dV]=low_solve(F_3,G_3,N);
        U=U+dU; V=V+dV;

        for j=2:N-1
            for i=2:N-1
                r=-(U(i,j)-U(i-1,j)+V(i,j)-V(i,j-1))*N-D(i,j);
                d=r*h/4;
                U(i,j)=U(i,j)+d;
                U(i-1,j)=U(i-1,j)-d;
                V(i,j)=V(i,j)+d;
                V(i,j-1)=V(i,j-1)-d;
```

```
                P(i,j)=P(i,j)+r;
                r=r/4;
                P(i-1,j)=P(i-1,j)-r;
                P(i+1,j)=P(i+1,j)-r;
                P(i,j-1)=P(i,j-1)-r;
                P(i,j+1)=P(i,j+1)-r;
            end
        end

        for i=2:N-1
            r=-(U(i,1)-U(i-1,1)+V(i,1))*N-D(i,1);
            d=r*h/3;
            U(i,1)=U(i,1)+d;
            U(i-1,1)=U(i-1,1)-d;
            V(i,1)=V(i,1)+d;
            P(i,1)=P(i,1)+r;
            r=r/3;
            P(i-1,1)=P(i-1,1)-r;
            P(i+1,1)=P(i+1,1)-r;
            P(i,2)=P(i,2)-r;
        end

        for i=2:N-1
            r=-(U(i,N)-U(i-1,N)-V(i,N-1))*N-D(i,N);
            d=r*h/3;
            U(i,N)=U(i,N)+d;
            U(i-1,N)=U(i-1,N)-d;
            V(i,N-1)=V(i,N-1)-d;
            P(i,N)=P(i,N)+r;
            r=r/3;
            P(i-1,N)=P(i-1,N)-r;
            P(i+1,N)=P(i+1,N)-r;
            P(i,N-1)=P(i,N-1)-r;
        end

        for i=2:N-1
            r=-(U(1,i)+V(1,i)-V(1,i-1))*N-D(1,i);
            d=r*h/3;
            U(1,i)=U(1,i)+d;
            V(1,i)=V(1,i)+d;
            V(1,i-1)=V(1,i-1)-d;
            P(1,i)=P(1,i)+r;
            r=r/3;
            P(2,i)=P(2,i)-r;
            P(1,i-1)=P(1,i-1)-r;
            P(1,i+1)=P(1,i+1)-r;
        end

```

```
        for i=2:N-1
            r=-(-U(N-1,i)+V(N,i)-V(N,i-1))*N-D(N,i);
            d=r*h/3;
            U(N-1,i)=U(N-1,i)-d;
            V(N,i)=V(N,i)+d;
            V(N,i-1)=V(N,i-1)-d;
            P(N,i)=P(N,i)+r;
            r=r/3;
            P(N-1,i)=P(N-1,i)-r;
            P(N,i-1)=P(N,i-1)-r;
            P(N,i+1)=P(N,i+1)-r;
        end

        r=-(U(1,1)+V(1,1))*N-D(1,1);
        d=r*h/2;
        U(1,1)=U(1,1)+d;
        V(1,1)=V(1,1)+d;
        P(1,1)=P(1,1)+r;
        r=r/2;
        P(2,1)=P(2,1)-r;
        P(1,2)=P(1,2)-r;

        r=-(U(1,N)-V(1,N-1))*N-D(1,N);
        d=r*h/2;
        U(1,N)=U(1,N)+d;
        V(1,N-1)=V(1,N-1)-d;
        P(1,N)=P(1,N)+r;
        r=r/2;
        P(2,N)=P(2,N)-r;
        P(1,N-1)=P(1,N-1)-r;

        r=-(-U(N-1,1)+V(N,1))*N-D(N,1);
        d=r*h/2;
        U(N-1,1)=U(N-1,1)-d;
        V(N,1)=V(N,1)+d;
        P(N,1)=P(N,1)+r;
        r=r/2;
        P(N-1,1)=P(N-1,1)-r;
        P(N,2)=P(N,2)-r;

        r=-(-U(N-1,N)-V(N,N-1))*N-D(N,N);
        d=r*h/2;
        U(N-1,N)=U(N-1,N)-d;
        V(N,N-1)=V(N,N-1)-d;
        P(N,N)=P(N,N)+r;
        r=r/2;
        P(N-1,N)=P(N-1,N)-r;
        P(N,N-1)=P(N,N-1)-r;
```

```
    end
end
```

这里对变量 $r$ 进行了复用, 避免了重复多次计算 $r/4, r/3, r/2$ , 能减少一些运算量; 同时由于 MATLAB 中矩阵元素的存储遵循列优先的原则 (与 C 语言相反), 因此逐格更新时二重循环先列后行, 这样能大幅提升缓存命中率, 显著提高元素的读取和存储速度, 对整体程序的运行速度有一定提升.

又注意到 DGS 更新这一步完全串行, 无法向量化并行加速, 是整个算法效率上的瓶颈, 因此笔者尝试使用 MATLAB 自带的 MEX 文件调用 C/C++ 程序的功能, 通过用 C/C++ 语言重写 DGS 更新这一函数来避免 MATLAB 解释器所导致的效率低下, 然而效果不甚理想, 原因可能是 MATLAB 和 C/C++ 存储矩阵的底层原理不同, 导致每次数据传输都要消耗大量时间. 或许可以通过工程上的手段让 MATLAB 程序和 C/C++ 程序始终原地修改同一个数据, 实现内存共享, 但笔者能力有限, 无法实现.

## 3.3 数值结果及分析

当 $N/L = 4$ 时, 在不同的 $\nu_1, \nu_2$ 和数据规模 $N$ 下的 V-cycle 次数和 CPU 时间见表 1 .

表 1: 第一题当 $N/L = 4$ 时, 在不同 $\nu_1, \nu_2$ 和数据规模 $N$ 下的 V-cycle 次数和 CPU 时间

| | $\nu_1 = 2, \nu_2 = 2$ | | $\nu_1 = 2, \nu_2 = 3$ | |
|---|---|---|---|---|
| 数据规模 $N$ | V-cycle 次数 | CPU 时间 | V-cycle 次数 | CPU 时间 |
| $N = 2^6$ | 11 | 0.019s | 7 | 0.014s |
| $N = 2^7$ | 11 | 0.040s | 7 | 0.032s |
| $N = 2^8$ | 11 | 0.13s | 7 | 0.10s |
| $N = 2^9$ | 11 | 0.83s | 7 | 0.69s |
| $N = 2^{10}$ | 11 | 4.3s | 7 | 3.4s |
| $N = 2^{11}$ | 11 | 18s | 7 | 15s |
| | $\nu_1 = 3, \nu_2 = 3$ | | $\nu_1 = 4, \nu_2 = 4$ | |
| 数据规模 $N$ | V-cycle 次数 | CPU 时间 | V-cycle 次数 | CPU 时间 |
| $N = 2^6$ | 7 | 0.015s | 6 | 0.016s |
| $N = 2^7$ | 7 | 0.034s | 6 | 0.037s |
| $N = 2^8$ | 7 | 0.11s | 6 | 0.11s |
| $N = 2^9$ | 7 | 0.72s | 6 | 0.77s |
| $N = 2^{10}$ | 7 | 3.7s | 6 | 4.0s |
| $N = 2^{11}$ | 7 | 16s | 6 | 18s |

当 $\nu_1 = 2, \nu_2 = 3$ 时, 在不同的 $L$ 和数据规模 $N$ 下的 V-cycle 次数和 CPU 时间见表 2 .

由此可知, V-cycle 的次数与 $\nu_1 + \nu_2, L$ 都呈现反相关的关系. 同时, 使得 CPU 时间最短的最佳参数设置为 $N/L = 4, \nu_1 = 2, \nu_2 = 3$ , 对应的在不同的数据规模 $N$ 下的计算误差 $e_N$ 见表 3 , 因此 $e_N = \Theta(N^{-2})$.

表 2: 第一题当 $\nu_1 = 2, \nu_2 = 3$ 时, 在不同的 $L$ 和数据规模 $N$ 下的 V-cycle 次数和 CPU 时间

| | $N/L = 4$ | | $N/L = 8$ | |
|---|---|---|---|---|
| 数据规模 $N$ | V-cycle 次数 | CPU 时间 | V-cycle 次数 | CPU 时间 |
| $N = 2^6$ | 7 | 0.014s | 16 | 0.030s |
| $N = 2^7$ | 7 | 0.032s | 16 | 0.070s |
| $N = 2^8$ | 7 | 0.10s | 15 | 0.21s |
| $N = 2^9$ | 7 | 0.69s | 15 | 1.5s |
| $N = 2^{10}$ | 7 | 3.4s | 14 | 6.8s |
| $N = 2^{11}$ | 7 | 15s | 14 | 29s |

表 3: 第一题当 $N/L = 4, \nu_1 = 2, \nu_2 = 3$ 时, 在不同的数据规模 $N$ 下的计算误差 $e_N$

| 数据规模 $N$ | 计算误差 $e_N$ | $N^2 e_N$ |
|---|---|---|
| $N = 2^6$ | $1.495 \times 10^{-3}$ | 6.124 |
| $N = 2^7$ | $3.736 \times 10^{-4}$ | 6.122 |
| $N = 2^8$ | $9.340 \times 10^{-5}$ | 6.121 |
| $N = 2^9$ | $2.335 \times 10^{-5}$ | 6.121 |
| $N = 2^{10}$ | $5.837 \times 10^{-6}$ | 6.121 |
| $N = 2^{11}$ | $1.459 \times 10^{-6}$ | 6.121 |

# 4 第二题

## 4.1 数值方法

采用 Uzawa Iteration Method, 并利用带状 $\boldsymbol{LDL}^{\mathrm{T}}$ 方法求解每一步的子问题 $\boldsymbol{AU}_{k+1} = \boldsymbol{F} - \boldsymbol{BP}_k$ , 其中 Uzawa Iteration Method (Hirofumi Uzawa 1958) 定义如下:

**1.** 求解 $\boldsymbol{AU}_{k+1} = \boldsymbol{F} - \boldsymbol{BP}_k$ ;

**2.** 更新压力 $\boldsymbol{P}_{k+1} = \boldsymbol{P}_k + \alpha \boldsymbol{B}^{\mathrm{T}} \boldsymbol{U}_{k+1}$ ;

**3.** 判断误差是否小于允许的值, 如果小于, 则停止迭代; 否则回到第一步.

带状 $\boldsymbol{LDL}^{\mathrm{T}}$ 方法采用基于带状矩阵的压缩存储方法, 即将原带状矩阵的带状区域完全存储在一个宽度为 $d+1$ ($d$ 为原矩阵的半带宽) 的矩阵中 (利用了原矩阵的对称性), 同时在后面的运算步骤中都以压缩后的矩阵为对象 (将原矩阵的索引变换为压缩后的矩阵的索引即可), 并且只需进行一次 $\boldsymbol{LDL}^{\mathrm{T}}$ 分解, 这样不仅显著提高了求解效率, 也完全解决了内存占用的问题.

参数 $\alpha$ 的选取: 因为

$$\boldsymbol{P}_{k+1} = \boldsymbol{P}_k + \alpha \boldsymbol{B}^{\mathrm{T}} \boldsymbol{A}^{-1} (\boldsymbol{F} - \boldsymbol{BP}_k) = (\boldsymbol{I} - \alpha \boldsymbol{B}^{\mathrm{T}} \boldsymbol{A}^{-1} \boldsymbol{B}) \boldsymbol{P}_k + \alpha \boldsymbol{B}^{\mathrm{T}} \boldsymbol{A}^{-1} \boldsymbol{F},$$

因此, 关于压力的更新方法, 相当于下面的线性方程组

$$\boldsymbol{B}^{\mathrm{T}} \boldsymbol{A}^{-1} \boldsymbol{BP} = \boldsymbol{B}^{\mathrm{T}} \boldsymbol{A}^{-1} \boldsymbol{F}$$

的迭代格式, 但注意到压力 $\boldsymbol{P}$ 在相差一个常数意义下唯一, 故无需考虑压力 $\boldsymbol{P}$ 在 $\boldsymbol{B}^{\mathrm{T}} \boldsymbol{A}^{-1} \boldsymbol{B}$ 的核空间 (即

特征值 0 对应的特征子空间) 上的投影分量, 因而由 2.2 节得最优参数为

$$\alpha^* = \frac{2}{\lambda^*_{\min}(\boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{-1}\boldsymbol{B}) + \lambda^*_{\max}(\boldsymbol{B}^{\mathrm{T}}\boldsymbol{A}^{-1}\boldsymbol{B})} = \frac{2}{1+1} = 1,$$

其中 $\lambda^*$ 表示非零特征值, 通过实际测试也可得到 $\alpha^* = 1$ 为最佳参数.

## 4.2 具体算法实现

```
function [U,V,count]=Uzawa_iteration(F,G,N,eps,a)
    A=initialize_A(N);
    A_L=banded_LDL_decomposition(A,N);
    count=0;
    r_0=sqrt(norm(F,'fro')^2+norm(G,'fro')^2);
    r=r_0;
    P=zeros(N,N);
    F_2=zeros(N-1,N);
    G_2=zeros(N,N-1);

    while r/r_0>eps
        count=count+1;

        F_new=(F-F_2).'/N^2;
        U_vec=banded_LDL_solve(A_L,F_new(:),N);
        U=reshape(U_vec,N,N-1).';

        G_new=(G-G_2)/N^2;
        V_vec=banded_LDL_solve(A_L,G_new(:),N);
        V=reshape(V_vec,N,N-1);
        D=BT_multiply(U,V,N);
        P=P+a*D;

        [F_1,G_1]=A_multiply(U,V,N);
        [F_2,G_2]=B_multiply(P,N);
        r=sqrt(norm(F-F_1-F_2,'fro')^2+norm(G-G_1-G_2,'fro')^2+norm(D,'fro')^2);
    end
end

function A_L=banded_LDL_decomposition(A,d)
    n=size(A,1);
    A_L=zeros(d+1,n);
    v=zeros(n,1);

    for j=1:n
        idx=j:min(n,j+d);
        A_L(idx-j+1,j)=A(idx,j);
    end

    for j=1:n
        i_1=max(1,j-d);
```

```
        i_2=min(n-j,d);
        s=0;

        for i=i_1:1:j-1
            a=A_L(j-i+1,i);
            v(i)=a*A_L(1,i);
            s=s+v(i)*a;
        end
        A_L(1,j)=A_L(1,j)-s;

        A_sub=zeros(i_2,j-i_1);
        for C=1:j-i_1
            C_1=i_1+C-1;
            C_2=j-C_1+1;
            for R=1:min(i_2,C_1+d-j)
               A_sub(R,C)=A_L(C_2+R,C_1);
            end
        end

        A_L(2:1+i_2,j)=(A_L(2:1+i_2,j)-A_sub*v(i_1:j-1))/A_L(1,j);
    end
end

function b=banded_LDL_solve(A_L,b,d)
    n=size(b);
    for j=1:n-1
        i_end=min(n,j+d);
        b(j+1:i_end)=b(j+1:i_end)-b(j)*A_L(2:i_end-j+1,j);
    end

    b=b./A_L(1,:).';
    for j=n:-1:2
        for i=max(1,j-d):j-1
            b(i)=b(i)-b(j)*A_L(j-i+1,i);
        end
    end
end
```

这里带状 $\boldsymbol{LDL}^{\mathrm{T}}$ 分解的具体算法实现中, 为了提高程序运行速度, 做了诸多优化, 包括二重循环先列后行, 向量化并行等.

## 4.3 数值结果及分析

当 $\alpha = 1$ 时, 在不同的数据规模 $N$ 下的计算误差 $e_N$ 见表 4 , 由此可见 $e_N = \Theta(N^{-2})$ .

表 4: 第二题当 $\alpha = 1$ 时, 在不同的数据规模 $N$ 下的计算误差 $e_N$

| 数据规模 $N$ | 计算误差 $e_N$ | $N^2 e_N$ |
|---|---|---|
| $N = 2^6$ | $1.495 \times 10^{-3}$ | 6.124 |
| $N = 2^7$ | $3.736 \times 10^{-4}$ | 6.122 |
| $N = 2^8$ | $9.340 \times 10^{-5}$ | 6.121 |
| $N = 2^9$ | $2.335 \times 10^{-5}$ | 6.121 |

# 5 第三题

## 5.1 数值方法

采用 Inexact Uzawa Iteration Method, 并以 V-cycle 多重网格方法为预条件子, 利用共轭梯度法求解每一步的子问题 $\boldsymbol{A}\boldsymbol{U}_{k+1} = \boldsymbol{F} - \boldsymbol{B}\boldsymbol{P}_k$ , 其中 Inexact Uzawa Iteration Method (Howard C Elman & Gene H. Golub 1994, SINUM, 1645-1661) 定义如下: 给定 $\boldsymbol{P}_0$ , 令 $k = 0$ ,

**1.** 近似求解 $\boldsymbol{A}\boldsymbol{U}_{k+1} = \boldsymbol{F} - \boldsymbol{B}\boldsymbol{P}_k$ , 得到近似解 $\widehat{\boldsymbol{U}}_{k+1}$ ;

**2.** 更新压力 $\boldsymbol{P}_{k+1} = \boldsymbol{P}_k + \alpha \boldsymbol{B}^{\mathrm{T}} \widehat{\boldsymbol{U}}_{k+1}$ ;

**3.** 判断误差是否小于允许的值, 如果小于, 则停止迭代; 否则回到第一步.

设 $\widehat{\boldsymbol{U}}_{k+1}$ 是方程 $\boldsymbol{A}\boldsymbol{U}_{k+1} = \boldsymbol{F} - \boldsymbol{B}\boldsymbol{P}_k$ 的近似解, 定义

$$\delta_k = \boldsymbol{A}\widehat{\boldsymbol{U}}_{k+1} - \boldsymbol{F} + \boldsymbol{B}\boldsymbol{P}_k,$$

若

$$\|\delta_k\|_2 \leqslant \tau \|\boldsymbol{B}^{\mathrm{T}} \widehat{\boldsymbol{U}}_k\|_2,$$

则 (适当选取 $\alpha$) 当 $\tau$ 充分小时, 上述迭代方法是收敛的.

预优共轭梯度法只需用到矩阵向量乘法, 由 2.5 节知可以高效运算.

## 5.2 具体算法实现

```
function [U,V,count]=inexact_Uzawa_iteration(F,G,N,eps,tau,a,v_1,v_2,L,k_m)
    r_0=sqrt(norm(F,'fro')^2+norm(G,'fro')^2);
    r=r_0;
    P=zeros(N,N);
    F_2=zeros(N-1,N);
    G_2=zeros(N,N-1);
    count=0;

    while r/r_0>eps
        count=count+1;
        [U,V]=PCG(F-F_2,G-G_2,N,eps,tau,v_1,v_2,L,k_m);
        D=BT_multiply(U,V,N);
        P=P+a*D;

        [F_1,G_1]=A_multiply(U,V,N);
```

```
        [F_2,G_2]=B_multiply(P,N);
        r=sqrt(norm(F-F_1-F_2,'fro')^2+norm(G-G_1-G_2,'fro')^2+norm(D,'fro')^2);
    end
end

function [U,V]=PCG(F,G,N,eps,tau,v_1,v_2,L,k_m)
    U=zeros(N-1,N);
    V=zeros(N,N-1);

    r_U=F;
    r_V=G;
    [z_U,z_V]=vcycle_3(r_U,r_V,N,eps,v_1,v_2,L);

    rho_U=dot(r_U(:),z_U(:));
    rho_V=dot(r_V(:),z_V(:));
    p_U=z_U;
    p_V=z_V;
    k=0;

    while 1
        D=BT_multiply(U,V,N);
        r=sqrt(norm(r_U,'fro')^2+norm(r_V,'fro')^2);
        if r<=tau*norm(D,'fro') || k>=k_m
            break;
        end

        k=k+1;
        [w_U,w_V]=A_multiply(p_U,p_V,N);
        alpha_U=rho_U/dot(p_U(:),w_U(:));
        alpha_V=rho_V/dot(p_V(:),w_V(:));
        U=U+alpha_U*p_U;
        V=V+alpha_V*p_V;
        r_U=r_U-alpha_U*w_U;
        r_V=r_V-alpha_V*w_V;
        [z_U,z_V]=vcycle_3(r_U,r_V,N,eps,v_1,v_2,L);
        rho_U1=dot(r_U(:),z_U(:));
        rho_V1=dot(r_V(:),z_V(:));
        beta_U=rho_U1/rho_U;
        beta_V=rho_V1/rho_V;
        rho_U=rho_U1;
        rho_V=rho_V1;
        p_U=z_U+beta_U*p_U;
        p_V=z_V+beta_V*p_V;
    end
end
```

### 5.3 数值结果及分析

当 $\alpha = 1, N/L = 4, \tau = 6 \times 10^{-5}$ 时, 在不同的 $\nu_1, \nu_2$ 和数据规模 $N$ 下的外循环次数和 CPU 时间见表 5 .

表 5: 第三题当 $\alpha = 1, N/L = 4, \tau = 6 \times 10^{-5}$ 时, 在不同 $\nu_1, \nu_2$ 和数据规模 $N$ 下的外循环次数和 CPU 时间

| | $\nu_1 = 2, \nu_2 = 2$ | | $\nu_1 = 2, \nu_2 = 3$ | |
|---|---|---|---|---|
| 数据规模 $N$ | 外循环次数 | CPU 时间 | 外循环次数 | CPU 时间 |
| $N = 2^6$ | 2 | 0.062s | 2 | 0.047s |
| $N = 2^7$ | 2 | 0.13s | 2 | 0.10s |
| $N = 2^8$ | 2 | 0.45s | 2 | 0.30s |
| $N = 2^9$ | 2 | 3.4s | 2 | 2.3s |
| $N = 2^{10}$ | 2 | 17s | 2 | 12s |
| $N = 2^{11}$ | 2 | 77s | 2 | 53s |
| | $\nu_1 = 3, \nu_2 = 3$ | | $\nu_1 = 4, \nu_2 = 4$ | |
| 数据规模 $N$ | 外循环次数 | CPU 时间 | 外循环次数 | CPU 时间 |
| $N = 2^6$ | 2 | 0.051s | 2 | 0.053s |
| $N = 2^7$ | 2 | 0.11s | 2 | 0.11s |
| $N = 2^8$ | 2 | 0.33s | 2 | 0.35s |
| $N = 2^9$ | 2 | 2.4s | 2 | 2.6s |
| $N = 2^{10}$ | 2 | 13s | 2 | 14s |
| $N = 2^{11}$ | 2 | 57s | 2 | 63s |

当 $\alpha = 1, \nu_1 = 2, \nu_2 = 3, \tau = 6 \times 10^{-5}$ 时, 在不同的 $L$ 和数据规模 $N$ 下的外循环次数和 CPU 时间见表 6 .

当 $\nu_1 = 2, \nu_2 = 3, N/L = 4, \tau = 6 \times 10^{-5}$ 时, 在不同的 $\alpha$ 和数据规模 $N$ 下的外循环次数和 CPU 时间见表 7 .

当 $\alpha = 1, \nu_1 = 2, \nu_2 = 3, N/L = 4$ 时, 在不同的 $\tau$ 和数据规模 $N$ 下的外循环次数和 CPU 时间见表 8.

表 6: 第三题当 $\alpha = 1, \nu_1 = 2, \nu_2 = 3, \tau = 6 \times 10^{-5}$ 时, 在不同的 $L$ 和数据规模 $N$ 下的外循环次数和 CPU 时间

| | $N/L = 4$ | | $N/L = 8$ | |
|---|---|---|---|---|
| 数据规模 $N$ | 外循环次数 | CPU 时间 | 外循环次数 | CPU 时间 |
| $N = 2^6$ | 2 | 0.047s | 2 | 0.10s |
| $N = 2^7$ | 2 | 0.10s | 2 | 0.22s |
| $N = 2^8$ | 2 | 0.30s | 2 | 0.69s |
| $N = 2^9$ | 2 | 2.3s | 2 | 5.1s |
| $N = 2^{10}$ | 2 | 12s | 2 | 26s |
| $N = 2^{11}$ | 2 | 53s | 2 | 160s |

表 7: 第三题当 $\nu_1 = 2, \nu_2 = 3, N/L = 4, \tau = 6 \times 10^{-5}$ 时, 在不同的 $\alpha$ 和数据规模 $N$ 下的外循环次数和 CPU 时间

| | $\alpha = 0.99$ | | $\alpha = 0.999$ | |
|---|---|---|---|---|
| 数据规模 $N$ | 外循环次数 | CPU 时间 | 外循环次数 | CPU 时间 |
| $N = 2^6$ | 4 | 0.10s | 3 | 0.074s |
| $N = 2^7$ | 4 | 0.22s | 3 | 0.16s |
| $N = 2^8$ | 4 | 0.67s | 3 | 0.49s |
| $N = 2^9$ | 4 | 5.4s | 3 | 3.7s |
| $N = 2^{10}$ | 4 | 29s | 3 | 18s |
| $N = 2^{11}$ | 4 | 120s | 3 | 85s |
| | $\alpha = 1$ | | $\alpha = 1.001$ | |
| 数据规模 $N$ | 外循环次数 | CPU 时间 | 外循环次数 | CPU 时间 |
| $N = 2^6$ | 2 | 0.047s | 3 | 0.075s |
| $N = 2^7$ | 2 | 0.10s | 3 | 0.16s |
| $N = 2^8$ | 2 | 0.30s | 3 | 0.48s |
| $N = 2^9$ | 2 | 2.3s | 3 | 3.7s |
| $N = 2^{10}$ | 2 | 12s | 3 | 18s |
| $N = 2^{11}$ | 2 | 53s | 3 | 85s |

表 8: 第三题当 $\alpha=1,\nu_1=2,\nu_2=3,N/L=4$ 时, 在不同的 $\tau$ 和数据规模 $N$ 下的外循环次数和 CPU 时间

| | $\tau=1\times10^{-6}$ | | $\tau=1\times10^{-5}$ | |
|---|---|---|---|---|
| 数据规模 $N$ | 外循环次数 | CPU 时间 | 外循环次数 | CPU 时间 |
| $N=2^6$ | 2 | 0.065s | 2 | 0.047s |
| $N=2^7$ | 2 | 0.14s | 2 | 0.14s |
| $N=2^8$ | 2 | 0.42s | 2 | 0.42s |
| $N=2^9$ | 2 | 3.2s | 2 | 3.2s |
| $N=2^{10}$ | 2 | 17s | 2 | 16s |
| $N=2^{11}$ | 2 | 75s | 2 | 75s |
| | $\tau=6\times10^{-5}$ | | $\tau=1\times10^{-4}$ | |
| 数据规模 $N$ | 外循环次数 | CPU 时间 | 外循环次数 | CPU 时间 |
| $N=2^6$ | 2 | 0.047s | 2 | 0.047s |
| $N=2^7$ | 2 | 0.10s | 2 | 0.10s |
| $N=2^8$ | 2 | 0.30s | 2 | 0.30s |
| $N=2^9$ | 2 | 2.3s | 2 | 2.3s |
| $N=2^{10}$ | 2 | 12s | 2 | 12s |
| $N=2^{11}$ | 2 | 53s | 2 | 53s |

由此可知, 外循环次数只与 $\alpha$ 有关, 当 $\alpha$ 越接近最佳值 $\alpha^*=1$ , 外循环次数越小, 并且当 $\alpha=1$ 时, 压力 $\boldsymbol{P}$ 的更新具有 “一步收敛” 性 (这由 2.2 节容易得到) . 同时, 使得 CPU 时间最短的最佳参数设置为 $\alpha=1,\nu_1=2,\nu_2=3,N/L=4,\tau=6\times10^{-5}$ , 对应的在不同的数据规模 $N$ 下的计算误差 $e_N$ 见表 9 , 因此 $e_N=\Theta(N^{-2})$ .

表 9: 第三题当 $\alpha=1,\nu_1=2,\nu_2=3,N/L=4,\tau=6\times10^{-5}$ 时, 在不同的数据规模 $N$ 下的计算误差 $e_N$

| 数据规模 $N$ | 计算误差 $e_N$ | $N^2e_N$ |
|---|---|---|
| $N=2^6$ | $1.495\times10^{-3}$ | 6.124 |
| $N=2^7$ | $3.736\times10^{-4}$ | 6.122 |
| $N=2^8$ | $9.340\times10^{-5}$ | 6.121 |
| $N=2^9$ | $2.335\times10^{-5}$ | 6.121 |
| $N=2^{10}$ | $5.837\times10^{-6}$ | 6.121 |
| $N=2^{11}$ | $1.459\times10^{-6}$ | 6.121 |

# 参考文献

[1] 胡俊. 多重网格方法简介 [EB/OL]. (2025-11-23).

[2] 许盛飏. 大作业 [EB/OL]. (2025-11-24).